Lab 1: RTL & CDC

Due End of Fall Quarter

Outline

The goal of this lab is to familiarize you with RTL to where we can give you a spec, and you should be able to implement it without too much difficulty. Your task is to implement a series of patterns & primitives, and reason through questions about them. If you have any difficulty, please reach out and ask for help!

Check-off

Please find Aki in the lab sometime (there will eventually be scheduled times to work on labs) to be checked off on the lab. The check-off will entail some conceptual questions and one simple SystemVerilog question.

Resources

ARC & ASIC Design Lectures (Lecture 1)

Section 1: RTL Design

1.1: Overlapping Sequence Detector

1.1.a: Implement the RTL to detect the sequence ‘10010’. Implement it as a Mealy machine. Matches may be used to start detecting a new sequence. The skeleton module and ports are given below:

```
module overlapping_seq_detector (
   input logic clk,
   input logic rst_n,
   input logic din, // input bit
   output logic detect // when 10010 is seen
);
```

1.1.b. Implement the RTL, but this time as a Moore machine. State the tradeoffs between Mealy and Moore.

1.1.c: How would you implement the module for a parameterizable sequence? What hardware data structure/concept (e.g. FIFO, LFSR, etc.) would you use? (bonus: implement it this way!)

1.2: Multiply-and-Accumulate (MAC) Unit

1.2.a. Implement the RTL for a multiply-and-accumulate unit. On asynchronous-low reset, your unit must reset its 12-bit accumulator. New inputs are provided every cycle. All outputs (i.e. the accumulator) must be registered.

```
module mac_unit (
   input logic clk,
   input logic rst_n,

   input logic [7:0] a, b,
   output logic [11:0] acc
);
```

1.2.b. Pipeline the MAC so that multiply is one cycle, and accumulate is another cycle.

1.3: Synchronous FIFO

1.3.a. Implement the Synchronous FIFO whose module interface was introduced during lecture. The consumer may only read if the FIFO is not empty (ignore the possibility of the producer adding data same-cycle), and vice-versa. You may assume power-of-two depth if you add an elaboration-time $fatal for it.

```
module sync_fifo #(
   parameter FIFO_DEPTH = 32,
   parameter DATA_W = 16
) (
   input logic clk,
   input logic rst_n,

   input logic in_valid,
   output logic in_ready,
   input logic [DATA_W-1:0] w_data,

   input logic out_valid,
   output logic out_ready,
   output logic [DATA_W-1:0] r_data
);
```

1.3.b. Remove the cycle of latency, and allow the consumer to consume from an ‘empty’ FIFO if the producer adds data in the same cycle, and vice versa.

1.3.c. Describe the tradeoff you make when adding support for same-cycle consuming and producing (hint: think about combinational paths across the module).

1.4: Arbitration, Resource Sharing

1.4.a. Implement the RTL for a fixed-priority arbiter, whose module interface is provided below. A fixed-priority arbiter accepts an N-bit request vector, and outputs an N-bit grant vector, always prioritizing from Requester 0 to Requester N-1. (hint: think about bit shifting and inversion tricks! write some examples)

```
module fixed_prio_arbiter #(
   parameter REQUESTERS = 8
) (
   input logic [REQUESTERS-1:0] reqs,
   output logic [REQUESTERS-1:0] grants,
   output logic grants_valid
);
```

1.4.b. By using two fixed-priority arbiters, you can implement a round-robin arbiter by using the idea of priority masking. One arbiter has (priority_mask & reqs), and one arbiter just accepts reqs. If the priority arbiter has no grants, use the general arbiter's grants. Implement a round-robin arbiter. (hint: for generating the priority mask, you can use grants and some bit shifting and inversion tricks as well…)

1.4.c. Explain the concept of 'starvation', and why round-robin arbiters solve the issue, while fixed-priority arbiters do not.

1.4.d. There's a second method of generating a round-robin arbiter (the code for which you can find in the verification lab, but will be summarized succinctly below). Describe the tradeoffs.

A round-robin arbiter can be generated by rotating the requester bit-vector a certain amount based on the last grant, and then passing that into a fixed-priority arbiter. After, the result grant is rotated backwards the same amount, and used as the output.

Section 2: Clock-Domain Crossing

2.1. Why not run everything on one clock? Explain CDC and metastability, and how the 2FF synchronizer solves many issues.

2.2. What doesn't the 2FF synchronizer solve, for example, if Clock A is 10x faster than Clock B, and Clock A is the producer?

2.3. Explain why the 2FF synchronizer cannot be used to transmit a bus of values. What measures do we take to transmit more than one bit at a time?

2.4. Implement an asynchronous FIFO, with the same module interface as the synchronous FIFO in 1.3, except with wclk and rclk. (hint: you must pass grey-coded pointers across the clock domain through 2FF synchronizers, then compare your local pointer to the stabilized grey-coded pointer. Why is comparing your new pointer to an old pointer legal?) **Assume pow-2 FIFO depth and add an elaboration-time check to your code.**

```
module async_fifo #(
   parameter FIFO_DEPTH = 32,
   parameter DATA_W = 16
) (
   input logic rclk, wclk,
   input logic rst_n,

   input logic in_valid,
   output logic in_ready,
   input logic [DATA_W-1:0] w_data,

   input logic out_valid,
   output logic out_ready,
   output logic [DATA_W-1:0] r_data
);
```

2.5. A consumer, operating at 100 MHz, writes into a synchronous FIFO for 100 cycles, then doesn't write for another 200 cycles, before repeating the cycle. The producer operates at 50 MHz. What depth is needed for the FIFO to prevent overflow? Assume they're not phase shifted.

2.6. Assume the synchronous FIFO becomes an asynchronous FIFO. What depth would you choose? Justify your reasoning.